Algorithm Selection Cheat Sheet

Find the first matching clue, then take the technique it points to. Italic phrases under each pill are the words to look for in the problem statement. Inspired by AlgoMonster's flowchart.

Graphs & trees

Tree structure?

- Level order or min depth?
  - Yes **BFS** — *shortest unweighted path · level order*
- Count or build tree shapes?
  - Yes **Divide & Conquer / Tree DP** — *count distinct trees · subtree DP*
  - No **DFS** — *tree paths · explore all branches*

Directed acyclic graph?
- Yes **Topological Sort** — *"course schedule" · prerequisites · DAG*

Shortest path?

- Weighted edges?
  - Yes **Dijkstra's Algorithm** — *shortest path · weighted edges*
  - No **BFS** — *shortest unweighted path · level order*

Connected components?
- Yes **Disjoint Set Union** — *connected components · "merge accounts"*

Tiny input?
- Yes **DFS / Backtracking** — *enumerate paths · small graph*
- No **BFS** — *shortest unweighted path · level order*

Small constraints (n is tiny)

Brute force viable?
- Yes **Brute Force / Backtracking** — *all permutations / subsets · n ≤ 12*

Track a subset?
- Yes **Bitmask DP** — *n ≤ 20 · "visit all" · subset state*
- No **Dynamic Programming** — *"number of ways" · max/min with choices*

Sorted, search & k-th

Sorted in, monotonic out?

- Range/rank queries + updates?
  - Yes **Ordered Set / Fenwick / Segment Tree** — *range sum + updates · order stats*
  - No **Binary Search** — *sorted array · "minimize the max"*

Top-k or k-th?
- Yes **Heap / Sorting** — *kth largest · "top k" · running median*

Linked lists

Linked list?

- Fast/slow or offset pointers?
  - Yes **Two pointers** — *pair from ends · fast/slow · in-place*
- Merge several sorted lists?
  - Yes **Heap / Divide & Conquer** — *merge k sorted lists*
  - No **Linked-List Manipulation** — *reverse / reorder a list*

Subarrays & windows

Running sums?
- Yes **Prefix Sums** — *subarray sum = k · negatives allowed · cumulative totals*

Contiguous span?

- Window invariant?
  - Yes **Sliding Window** — *longest/shortest substring · ≤ k distinct · exactly k → atMost(k)−atMost(k−1)*
- Nearest greater/smaller?
  - Yes **Monotonic Stack** — *next greater · "daily temperatures"*
  - No **Dynamic Programming** — *"number of ways" · max/min with choices*

Lookups, intervals, partition, strings

Lookup, count, or group?
- Yes **Hash Table / Counting** — *frequency · "first unique" · group by*

Overlapping intervals?
- Yes **Sorting + Interval Scan** — *merge intervals · "meeting rooms"*

Partition in place?
- Yes **Two pointers / Partitioning** — *move zeroes · Dutch flag · in-place*

Match/split via dictionary?

- Prefix/pattern search?
  - Yes **Trie / String Matching / Rolling Hash** — *prefix search · "starts with"*
  - No **Trie / DP / Memoization** — *word break · dictionary split*

Counting arrangements

Brute force viable?
- Yes **Brute Force / Backtracking** — *all permutations / subsets · n ≤ 12*
- No **Dynamic Programming** — *"number of ways" · max/min with choices*

Several sequences?

- Monotonic property?
  - Yes **Two pointers** — *pair from ends · fast/slow · in-place*
- Overlapping subproblems?
  - Yes **Dynamic Programming** — *"number of ways" · max/min with choices*

Locate indices?

- Monotonic property?
  - Yes **Two pointers** — *pair from ends · fast/slow · in-place*

Constant space?

- Monotonic property?
  - Yes **Two pointers** — *pair from ends · fast/slow · in-place*

Compute a max / min

Monotonic search space?
- Yes **Binary Search** — *sorted array · "minimize the max"*

Nearest greater/smaller?
- Yes **Monotonic Stack** — *next greater · "daily temperatures"*

Overlapping subproblems?
- Yes **Dynamic Programming** — *"number of ways" · max/min with choices*
- No **Greedy Algorithms** — *intervals · "jump game" · local choice*

Parsing, design, simulation, math (the tail)

Parsing symbols?

- Count or optimise segments?
  - Yes **Dynamic Programming** — *"number of ways" · max/min with choices*
  - No **Stack** — *valid parentheses · expression parse*

Design a structure?
- Yes **Design + Supporting Structures** — *"implement LRU / iterator"*

Just follow the rules?
- Yes **Simulation / Basic DSA** — *follow the rules step by step*

Math or bit tricks?

- Properties of numbers?
  - Yes **Number Theory** — *gcd / primes / modular arithmetic*
  - No **Math / Bit Manipulation** — *XOR tricks · powers · "single number"*

Otherwise
- · **Specialized / Advanced Pattern** — *rare / problem-specific trick*

FAMILY graph/traversal · search · pointers/window · heap/sort/interval · dynamic prog. · brute/backtrack · stack · other

Yes (green) take it · No (orange) alternative · purple = grouping question · · = fall-through default